COSC-4117EL Assignment 2

Warehouse Robot Decision Making with
Dynamic Programming and Reinforcement Learning

Group 1 (G1)
Group members: [fill in names / student IDs]

Fixed layout: seed 99 (Pickup P = (8, 0), Packing D = (7, 7), Charging dock C = (4, 2))
Discount factor $\gamma = 0.95$

1. Introduction and MDP formulation (Part 1, Q1)

A warehouse robot starts on the charging dock (green cell C), must first collect a package from the pickup station (blue P), deliver it to the packing station (yellow D) while carrying it, and finally return to the charging dock. Black shelves block movement and red congestion zones can be entered but incur a penalty. The experiments use the fixed seeded layout (seed 99) shown in Figure 1 of the brief; the provided environment was only extended to implement the stochastic motion model (documented in the file header).

1.1 State space S

The state is the tuple **s = (row, col, has_package, delivered)**, where row, col $\in$ {0,...,9} and the two task flags are binary. Position (x, y) alone is **not** sufficient: the reward received on the packing cell and on the dock and the set of optimal actions depend on the task phase. For example, standing on the dock with no package is an ordinary state, whereas standing on the dock after delivery is terminal. Adding the two binary flags makes the future depend only on the current state (the Markov property). The combination (has_package=0, delivered=1) is logically impossible and is excluded. Shelf cells are excluded as well. For the seed-99 layout there are 77 passable cells, giving 231 reachable MDP states (three task phases each), of which 1 is the success terminal state.

1.2 Action space A

A = {up, down, left, right}. All four actions are available everywhere; a move that would enter a shelf or leave the grid leaves the robot in the same cell and incurs the invalid-move penalty (rather than being removed from the action set).

1.3 Transition model P(s'|s,a)

When the robot selects an intended action a, the action occurs with probability p; the remaining probability 1-p is split equally between the two perpendicular directions (no backwards move). For Parts 1-2, p = 0.80, so e.g. choosing up yields up with 0.8, left with 0.1 and right with 0.1. If a realised movement would enter a shelf or leave the grid, the robot stays in place. Part 3 also evaluates p = 0.90 and p = 1.00.

1.4 Reward model R(s,a,s')

Event	Reward
Pick up the package at the pickup station	+25
Deliver the package at the packing station while carrying	+40
Return to the charging dock after delivery	+100
Enter a congestion zone (red cell)	-10
Every time step	-1
Attempted move blocked by a shelf / grid edge (realised)	-3

Rewards are additive; for example a realised blocked step costs -1 + (-3) = -4, and a blocked step that lands on a congestion cell is impossible by construction (the robot stays put).

1.5 Discount factor and terminal conditions

The discount factor is γ **= 0.95**, which keeps the task valuable (all goal rewards can be earned in tens of steps) while still preferring shorter routes. The **success terminal** condition is being on the charging dock with delivered = 1 (the +100 reward is paid and the episode ends). The 120-step limit is a simulation horizon (an episode ending with 'max_steps' is counted as a failure), not a stationary MDP state; Value

Iteration therefore models only the success terminal, while the simulator applies the step limit.

2. Value Iteration (Part 1, Q2)

The known transition and reward models are enumerated directly from the environment's pure transition kernel (the same function the simulator uses, so the planner's model cannot disagree with the environment). Value Iteration repeatedly applies the Bellman optimality update $V_{k+1}(s) = \max_a \sum_{s'} P(s'|s,a)\,[R(s,a,s') + \gamma\, V_k(s')]$ over all non-terminal states, starting from $V_0 = 0$, until $\max_s |V_{k+1}(s) - V_k(s)| < \theta$, with θ **$= 10^{-6}$**. The greedy policy is then extracted as argmax_a of the Bellman backup.

Intended-action p	Sweeps to converge	Final max residual	V at start state
1.00	38	0 (exact)	50.48
0.90	68	6.83e-07	37.62
0.80	86	9.15e-07	23.51

For the Part 1 setting (p = 0.80) Value Iteration converged in **86 sweeps**; convergence is slower as motion becomes noisier (38 sweeps at p = 1.00, 68 at p = 0.90, 86 at p = 0.80), because stochastic transitions spread value information over more neighbouring states per backup.

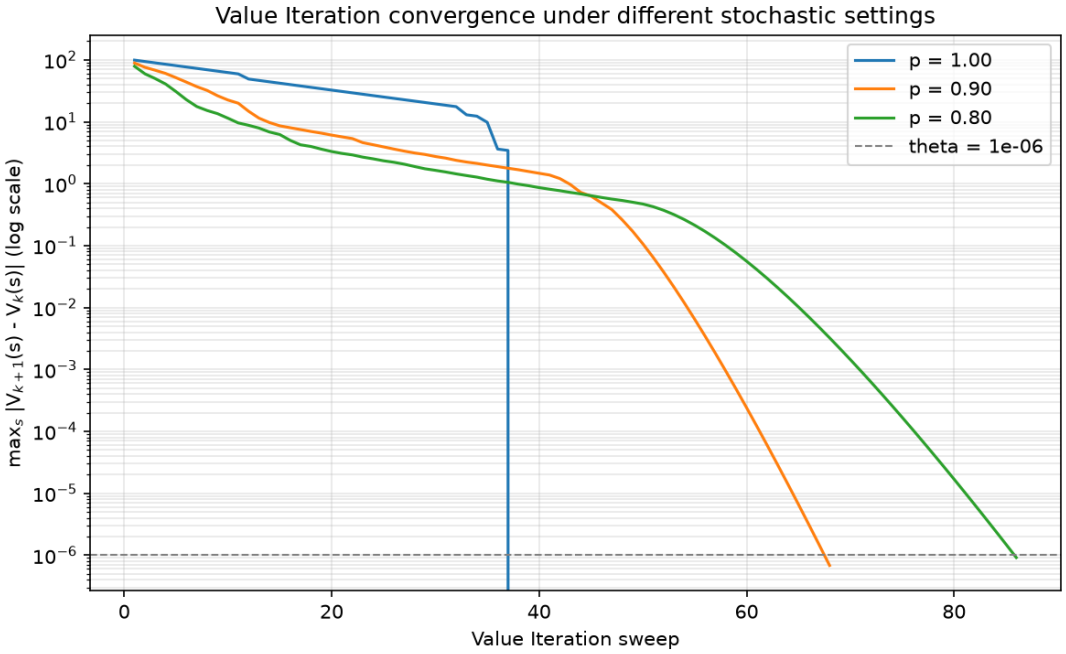


Figure 1. Maximum Bellman residual per sweep (log scale). More stochastic motion needs more sweeps to reach theta = 1e-6.

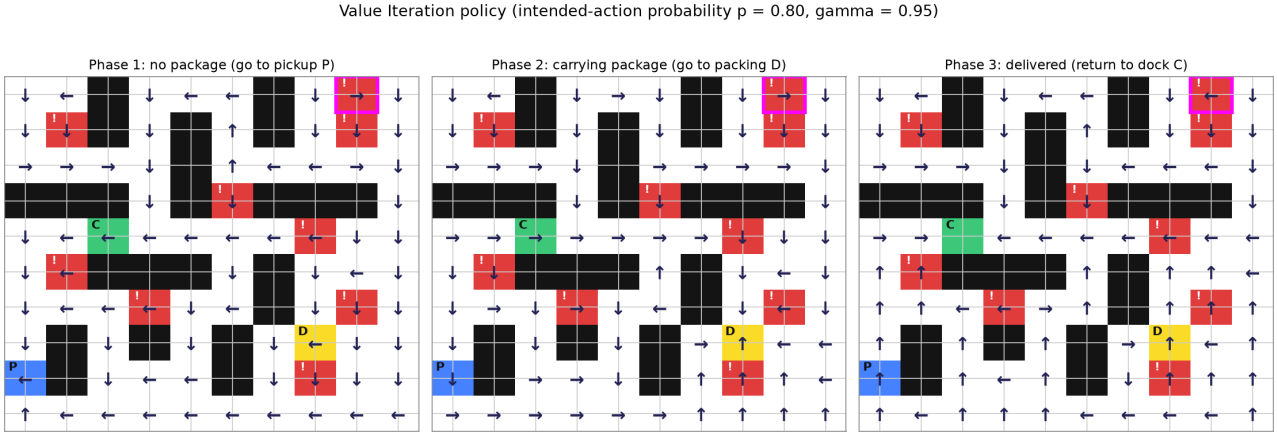


Figure 2. Value Iteration policy for p = 0.80 in the three task phases: go to pickup (left), carry to packing (middle), return to dock (right). The magenta square marks a state whose preferred action changes as p decreases (analysed in Section 4.4).

The policy is intuitively correct: in phase 1 it routes down the western edge to P, in phase 2 it crosses the bottom corridor and comes up column 6/7 to D, and in phase 3 it returns along row 4/5 to C. It deliberately uses one congestion cell ((3,5)) on the return leg because the deterministic shortest detour around it costs more than the -10 penalty.

Why a known model is required. Every Bellman backup explicitly sums over all possible successor states using P(s'|s,a) and R(s,a,s'). Without the model the expected values of untried actions could not be evaluated without sampling them, which is exactly why Part 2 needs a model-free learning algorithm.

3. Q-Learning (Part 2)

A Q-table of size 400 x 4 (state index x four actions) is initialised to zero. The agent acts epsilon-greedy, with epsilon decayed geometrically each episode from 1.0 to a floor of 0.01, and applies the standard update $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma(1\text{-done})\max_{a'}Q(s',a') - Q(s,a)]$. The agent never accesses $P(s'|s,a)$; it learns only from sampled experience. Two hyperparameter configurations were compared over three training seeds (42, 7, 123), using the fixed seed-99 layout:

Config	alpha	gamma	eps decay	eps range	episodes
A (conservative)	0.1	0.95	0.9995	1.00 -> 0.01	8000
B (fast adapting)	0.5	0.99	0.999	1.00 -> 0.01	8000

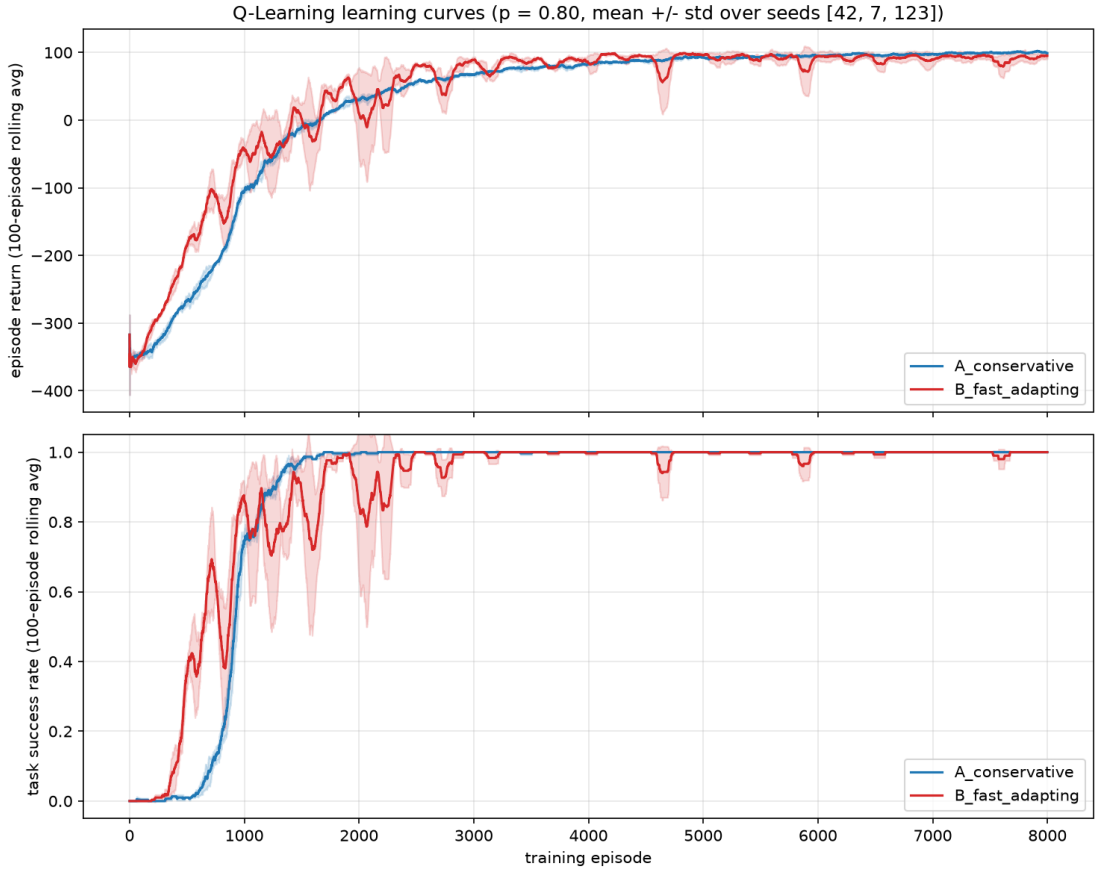


Figure 3. Training curves at p = 0.80 (100-episode rolling mean; shaded band = +/- standard deviation across seeds 42, 7, 123). Top: episode return; bottom: task success rate.

Both configurations learn the task: the rolling success rate approaches 1.0 after roughly 1.3k-2k episodes and the return approaches the Value Iteration level. Config B (large alpha 0.5, high gamma 0.99, faster exploration decay) learns faster early but is noisier and less robust: for seed 123 its fully greedy policy becomes brittle (a 1%-greedy exploration rate during training masks cycles that pure greedy execution then falls into), giving an evaluation success rate of only 0.50. Config A (smaller alpha, slower decay) is smooth and succeeds on all three seeds.

Config	seed	avg return	success rate	avg steps
A	42	101.28	1.000	32.1
A	7	93.84	1.000	34.3
A	123	101.22	1.000	32.1
A	mean +/- std	98.78 +/- 3.49	1.000	32.8

B	42	100.21	1.000	32.6
B	7	91.50	1.000	32.9
B	123	-37.92	0.504	95.4
B	mean +/- std	51.26 +/- 63.16	0.835	53.6

Config A is therefore chosen for the Part 3 comparison. After training the greedy policy was extracted and evaluated with exploration disabled (500 episodes, identical evaluation seeds for both methods). Figure 4 shows the resulting policy and Figure 5 one successful greedy run (the orange rings mark the pickup and delivery events).

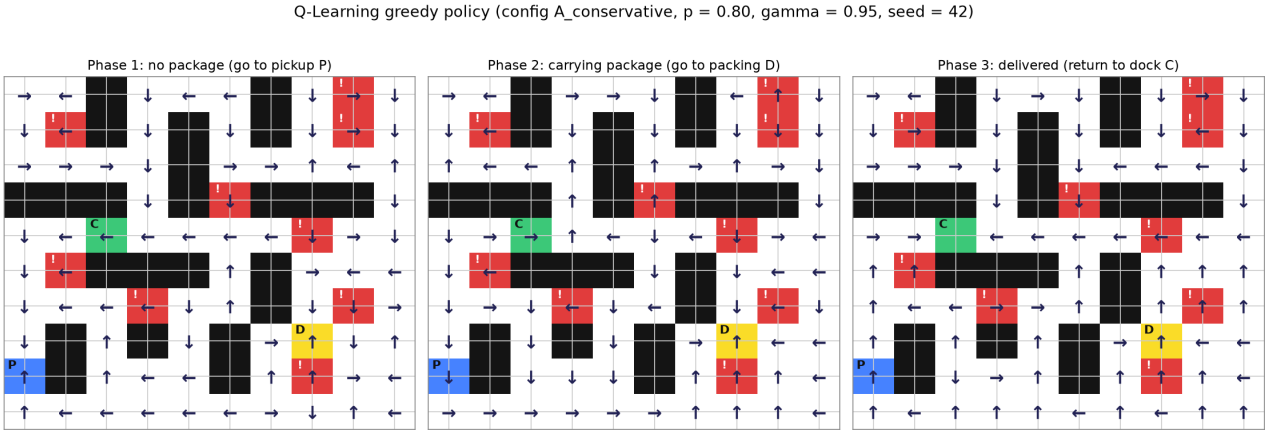


Figure 4. Q-Learning greedy policy (config A, best seed 42) at p = 0.80.

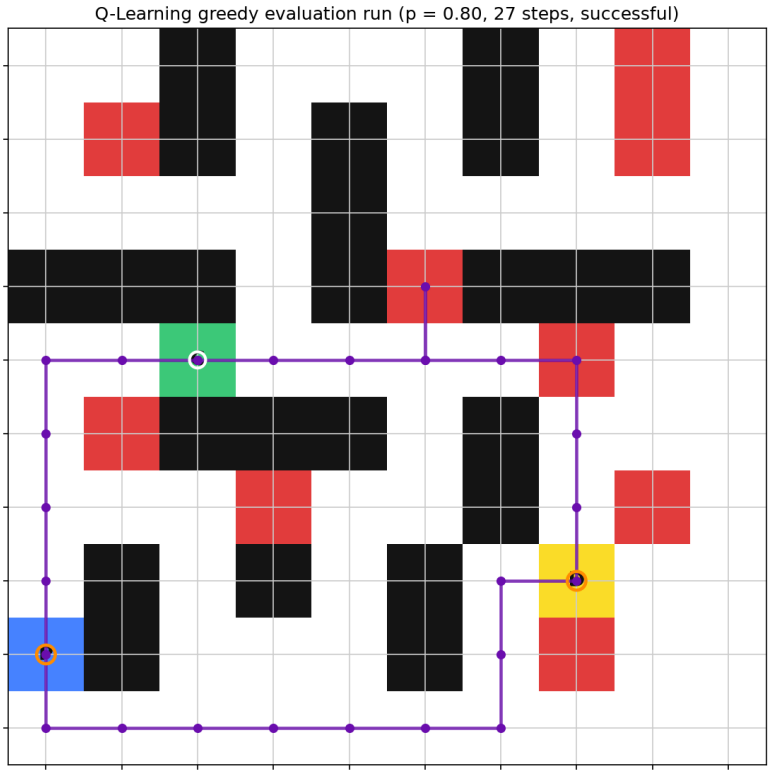


Figure 5. One successful greedy evaluation run: dock -> pickup -> packing -> dock in 27 steps at p = 0.80.

3.1 Comparison with the Value Iteration policy

At p = 0.80 both methods reach the same performance (Section 4), but the Q-Learning and Value Iteration arrows agree on only about 72% of all grid cells. Restricting the comparison to states the trained agent

actually visits during greedy evaluation, agreement rises to about 94% (100% at p = 1.00). The remaining whole-grid differences are harmless: Q-Learning rarely visits states far from the successful corridors, so their Q-values stay near zero and argmax picks arbitrary actions, and some near-tied routes (equal expected return) are resolved differently. The two methods learn the same behaviour where the behaviour matters.

4. Stochastic environment, comparison and visualisation (Part 3)

4.1 Quantitative comparison under p = 1.00 / 0.90 / 0.80

p	VI sweeps	VI return	QL return (3 seeds)	success VI / QL	VI steps	QL steps	QL converge episod
1.00	38	131.00	131.00 +/- 0.00	1.000 / 1.000	24.0	24.0	899
0.90	68	118.36	118.35 +/- 0.01	1.000 / 1.000	27.5	27.5	951
0.80	86	101.36	101.36 +/- 0.17	1.000 / 1.000	32.1	32.1	1282

*Median training seed; first episode after which the 100-episode rolling success rate stays at least 0.9.

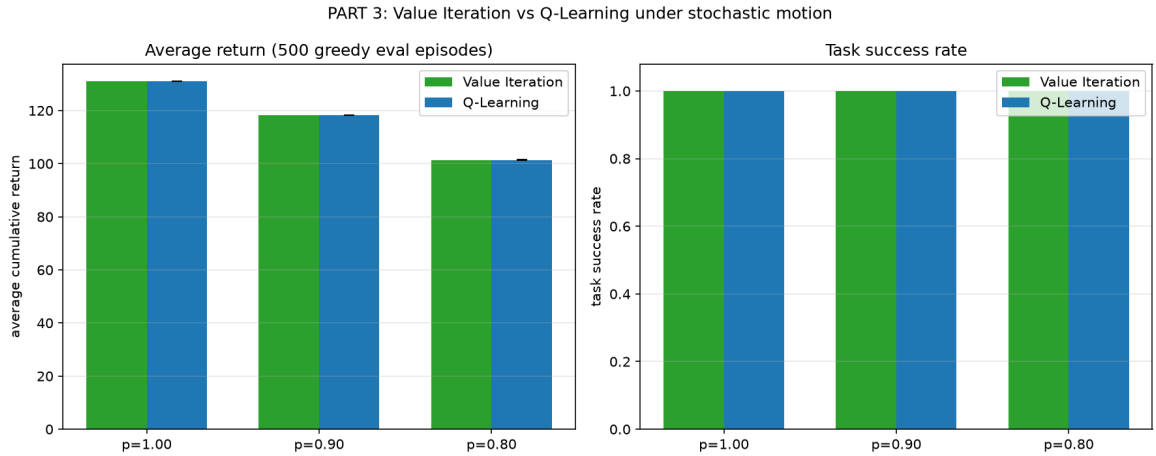


Figure 6. Average cumulative return and task success rate over 500 greedy evaluation episodes for both methods.

Q-Learning matches the model-based optimum almost exactly at every p (differences below 0.02 reward). Both methods always complete the task, but average return drops from 131 to 101 and path length grows from 24 to about 32 steps as p decreases: noise causes slips, blocked moves and extra congestion entries. Value Iteration needs more Bellman sweeps under noise, while Q-Learning needs more interaction episodes (about 900 at p = 1.00 vs about 1280 at p = 0.80).

4.2 Route behaviour near congestion zones

p	mean congestion entries (VI / QL)	mean blocked moves (VI / QL)	mean path length (VI / QL)
1.00	1.00 / 1.00	0.00 / 0.00	24.0 / 24.0
0.90	1.44 / 1.44	1.57 / 1.56	27.5 / 27.5
0.80	2.06 / 2.02	3.63 / 3.64	32.1 / 32.3

Two effects are visible. First, the **optimal policy changes** to avoid corridors that are risky under slipping (Section 4.4) - for example, preferring the open eastern column over the shelf-pinched western side when in the top-right congestion cluster. Second, even with rerouting, random slips still push the robot into red cells, so realised congestion entries (1.0 -> 1.44 -> 2.06) and blocked moves (0 -> 1.6 -> 3.6) increase with uncertainty. The learned Q-Learning policies reproduce the same routing behaviour and statistics.

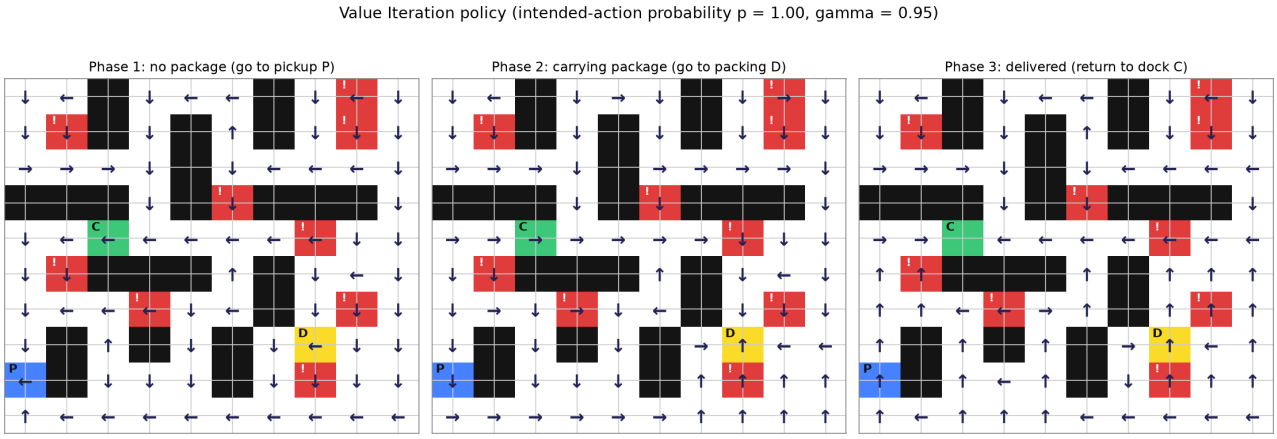


Figure 7. Value Iteration policy at p = 1.00 (deterministic).

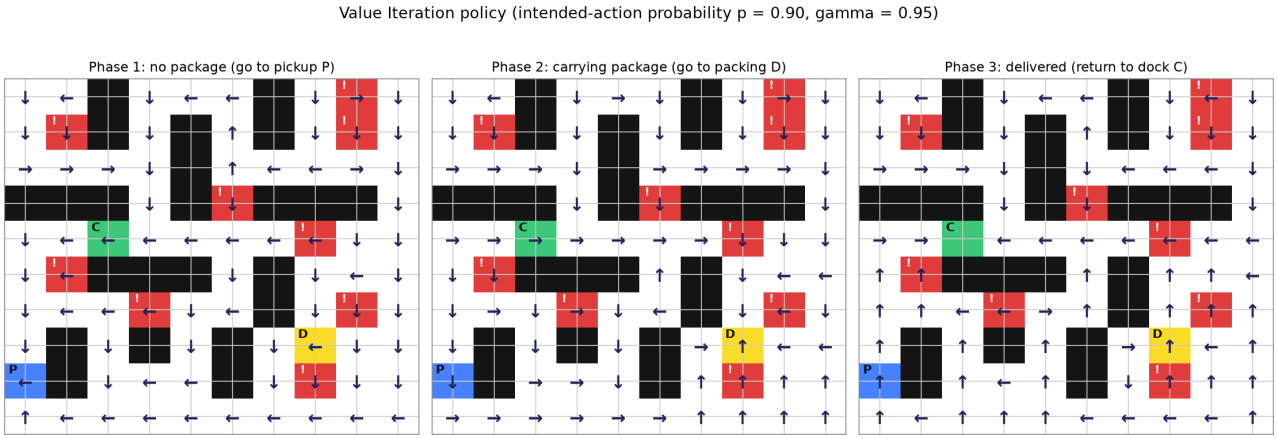


Figure 8. Value Iteration policy at p = 0.90.

4.3 States whose preferred action changes

Comparing the p = 1.00 and p = 0.80 Value Iteration policies, **15 states** change their preferred action, 5 of which are on or immediately adjacent to a congestion cell. Examples include (0,8,0,0) and (2,8,0,0) changing left -> right, (2,5,0,0) changing down -> up, and (5,1,0,0) changing down -> left.

4.4 Worked example: state (0, 8, 0, 0)

This state sits on the congestion cell in the top-right corner during phase 1. The one-step outcomes of moving left or right are symmetric (0.8 step to the neighbouring cell, 0.1 blocked slip at the top edge, 0.1 slip down into congestion (1,8)), so the difference is driven by the value of the continuation:

p	Q(left)	Q(right)	preferred	V(0,7)	V(0,9)
1.00	14.46	14.37	left	16.27	16.18
0.90	1.26	1.62	right	-	-
0.80	-12.22	-11.79	right	-9.41	-8.85

At p = 1.00 the western continuation is marginally better. At p = 0.80 the order flips: column 9 is an open straight corridor (cells (1,9), (2,9), (3,9) are all empty, so a slip rarely causes a penalty), whereas the western route through column 7 is squeezed between shelves at (0,6)/(1,6) and the row-3 shelf block (columns 6-8 are shelves), forcing a left turn while a downward slip can be blocked (-3) or bounce the robot back towards the congestion cell. With 20% slip probability the safer eastern continuation wins, so the preferred action changes from left to right. This is exactly the risk-averse rerouting the Bellman backup produces when the transition model is stochastic.

5. Conclusion

- The warehouse task is an MDP with state (row, col, has_package, delivered); position alone violates the Markov property.
- Value Iteration with gamma = 0.95 and theta = 1e-6 converged in 86 sweeps at p = 0.80 and yields the optimal policy directly from the known model.
- Model-free Q-Learning (alpha = 0.1, gamma = 0.95, epsilon decayed 1.0 -> 0.01) reaches the same performance over three seeds, with learning converging around 1.3k-2k episodes; a more aggressive configuration learns faster but is less robust to seed effects.
- Under p = 1.00 / 0.90 / 0.80 both methods achieve a 100% success rate, with average return 131.00 / 118.36 / 101.36 and mean path length 24.0 / 27.5 / 32.1. QL and VI agree on essentially all states the learned policy actually visits.
- Increasing uncertainty changes the preferred route near congestion and wall pinch points: 15 states flip actions, including (0,8,0,0) where the policy switches from the shelf-pinched western side to the open eastern corridor.

6. Generative AI use statement

The original environment template header states that it was generated with the assistance of ChatGPT. For this submission, the Doubao (ByteDance) generative AI assistant was used to: (i) extend the environment with the stochastic transition kernel; (ii) draft and debug the Value Iteration and Q-Learning implementations and the plotting code; (iii) draft the report text. All code was reviewed, run and verified by the group, and every numerical result in this report was produced by the submitted scripts. The following prompts were used (please paste the share links generated by the tool before submission):

- Implement stochastic transitions (intended action p, perpendicular slips (1-p)/2) and a pure transition kernel for the provided warehouse gridworld, and explain the MDP state representation.
- Implement Value Iteration with the Bellman optimality update, convergence threshold, greedy policy extraction and arrow visualisations for the three task phases.
- Implement tabular Q-Learning with epsilon-greedy decay, two alpha/gamma configurations, multiple seeds, learning curves and greedy evaluation; compare it with the Value Iteration results under p = 1.00/0.90/0.80.
- Explain why the preferred action changes at state (0,8,0,0) as p decreases, and draft the report sections.

7. Group member contribution statement

[Fill in: contributions of each member. All members confirm that the work is submitted in accordance with the Student Academic Integrity policy. No grade adjustment is requested / if one is requested, describe the situation and attach the agreement of all members.]